Лабораторная работа №15

Создание и использование строковых ресурсов

Основные правила

1. Не бойтесь **задавать любые вопросы**
2. Если хотите сделать что-то свое - **предлагайте**, будем думать
3. Советую читать текст: здесь написано много полезного

О чем лабораторная?

Текст в Android-приложении можно записать прямо в XML-разметке или Java-коде. Но если одна и та же надпись встречается в нескольких местах, ее изменение превращается в поиск по проекту. А если все тексты разбросаны по разным файлам, становится сложнее поддерживать интерфейс.

В Android для текстов предусмотрены **строковые ресурсы**. Они хранятся в файле `res/values/strings.xml`, а разметка и Java-код обращаются к ним по имени.

Сегодня мы перенесем тексты небольшого приложения в `strings.xml`, используем ресурсы в XML и Java, а затем проверим, что изменение одного ресурса обновляет все места, которые на него ссылаются.

Часть 1. Смотрим, в чем проблема

Разместите на экране:

- заголовок "Мои заметки"
- надпись "Количество заметок: 3"
- кнопку "Добавить заметку"
- кнопку "Очистить"

На этом этапе допустимо записать текст непосредственно в атрибутах `android:text`.

Теперь представьте, что название "Мои заметки" нужно изменить на "Мой блокнот" сразу в нескольких местах: в заголовке экрана и в кнопке или сообщении. Если текст везде записан отдельно, придется найти и исправить каждое название - при этом легко пропустить одно из них.

> Подумайте, какие надписи могут повторяться и что придется делать, если их потребуется изменить?

Часть 2. Создаем строковые ресурсы

Откройте файл:

`res/values/strings.xml`

Если в нем уже есть ресурс `app_name`, **не удаляйте его**. Добавьте внутрь тега `<resources>` строки для интерфейса.

Например:

```
<resources>
    <string name="app_name">Мои заметки</string>
    <string name="notes_count">Количество заметок: 3</string>
    <string name="add_note">Добавить заметку</string>
    <string name="clear">Очистить</string>
</resources>
```

У каждого ресурса есть два важных элемента:

- `name` - имя, по которому к ресурсу обращаются в коде
- **текст между открывающим и закрывающим тегами** - значение, которое увидит пользователь

Имя ресурса задается в коде латинскими буквами, цифрами и подчеркиваниями. Обычно используют понятные имена в нижнем регистре, например `add_note` или `notes_count`.

> **Самостоятельное задание:** добавьте в `strings.xml` ресурсы для всех надписей своего экрана. Не создавайте два разных ресурса для одной и той же надписи без необходимости.

Часть 3. Подключаем ресурсы в XML-разметке

Чтобы использовать строковый ресурс в XML, укажите ссылку вида:

```
android:text="@string/add_note"
```

Например:

```
<Button
    android:id="@+id/btnAdd"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/add_note" />
```

Android найдет ресурс `add_note` в `strings.xml` и подставит его значение.

Замените прямые текстовые значения в атрибутах `android:text` ссылками на созданные ресурсы. Не меняйте идентификаторы элементов: `android:id` нужен для обращения к View, а `@string/...` - для получения текста.

> **Самостоятельное задание:** перенесите все надписи из XML-разметки в `strings.xml`. Запустите приложение и проверьте, что внешний вид не изменился.

Часть 4. Проверяем главное преимущество

Теперь измените значение ресурса `add_note`:

```
<string name="add_note">Создать заметку</string>
```

Запустите приложение. Кнопка должна отображать новый текст, **хотя ее XML-разметку** вы не меняли.

Если один и тот же ресурс используется несколькими элементами, изменение его значения отразится во всех этих местах.

Это и есть одна из главных причин использовать строковые ресурсы: **текст хранится в одном месте, а разные части интерфейса обращаются к нему по имени**.

> **Задание:** используйте один и тот же строковый ресурс в двух разных элементах интерфейса.

Часть 5. Используем строковый ресурс в Java

Ресурсы можно получать не только в XML-разметке, но и из Java-кода.

Например, в `MainActivity.java`:

```
String message = getString(R.string.add_note);
```

Метод `getString()` возвращает значение строкового ресурса. Теперь его можно использовать, например, при нажатии на кнопку:

```
btnAdd.setOnClickListener(v -> {
    Toast.makeText(
        this,
        getString(R.string.add_note),
        Toast.LENGTH_SHORT
    ).show();
});
```

В этом примере кнопка показывает короткое сообщение с текстом из ресурса.

Обратите внимание: `R.string.add_note` - это идентификатор ресурса, а `getString(R.string.add_note)` - способ получить его текст.

> **Самостоятельное задание:** добавьте обработчик одной из кнопок. При нажатии покажите `Toast`, используя строковый ресурс.

Часть 6. Проверяем результат

Выполните проверки:

1. Все надписи интерфейса берутся из `strings.xml`.
2. После изменения значения ресурса текст в интерфейсе обновляется.
3. Один ресурс можно использовать в нескольких местах.
4. Кнопка показывает сообщение, полученное через `getString()`.
5. Приложение запускается без ошибок.

Ответьте на вопросы:

- Чем `android:id` отличается от `@string/...`?
- Что нужно изменить, если текст ресурса поменялся?
- Почему удобнее изменить один общий ресурс, чем искать одинаковый текст по нескольким файлам?
- Что произойдет, если в XML указать `@string/add_nots`, а ресурс называется `add_note`?

Самостоятельное задание

Добавьте на экран надпись "Последнее действие: нет". При нажатии на кнопку "Добавить заметку" меняйте ее на "Последнее действие: добавление", а при нажатии на кнопку "Очистить" - на "Последнее действие: очистка".

Требования

1. Создан или дополнен файл `res/values/strings.xml`
2. Текстовые надписи интерфейса вынесены в строковые ресурсы
3. В XML-разметке используются ссылки вида `@string/...`
4. В Java-коде используется `getString()` для получения текста
5. Один строковый ресурс используется минимум в двух местах интерфейса
6. Изменение значения ресурса отражается во всех местах, где он используется
7. Кнопки показывают сообщения с использованием строковых ресурсов
8. Приложение запускается без ошибок

Дополнительно

1. Создайте форматированную строку с подстановкой имени, например "Здравствуйте, %1$s!", и получите ее с помощью `getString(R.string.welcome, userName)`
2. Создайте ресурс для сообщения с числом, например количеством заметок, используя форматирование строки вместо хранения всего сообщения в Java-коде
3. Найдите предупреждения Android Studio о текстах, непосредственно записанных в XML, и исправьте оставшиеся случаи, если они есть